



IBM Developer
SKILLS NETWORK

Winning Space Race with Data Science

Abdul Rauf
20 August 2025



Outline

- Executive Summary
- Introduction
- Methodology
- Results
- Conclusion

Executive Summary

In this capstone project, our objective is to **predict the successful landing of the SpaceX Falcon 9 first stage** using various machine learning classification techniques.

The project workflow includes:

- **Data collection, cleaning, and preparation**
- **Exploratory data analysis (EDA)** to uncover patterns
- **Interactive visualizations** for deeper insights
- **Model training and evaluation** for prediction

Our analysis indicates that certain launch features — such as payload mass, orbit type, and launch site — show a strong correlation with the landing outcome.

Based on performance results, the **Decision Tree classifier** emerged as the most promising model for predicting Falcon 9 first-stage landing success.

Introduction

- In this capstone project, our goal is to **predict whether the Falcon 9 first stage will land successfully**. SpaceX offers Falcon 9 launches at around **\$62 million** per mission, compared to more than **\$165 million** charged by other providers. A major reason for this cost advantage is SpaceX's ability to **reuse the first stage**. By accurately predicting landing outcomes, we can estimate the potential cost of a launch, which could be valuable for other companies looking to compete with SpaceX in bidding for launch contracts.
- It's worth noting that some unsuccessful landings are intentional — for example, controlled descents into the ocean.

The core question we aim to answer is:

- *Given a set of features for a Falcon 9 launch — such as payload mass, orbit type, and launch site — can we accurately predict whether the first stage will land successfully?*

Section 1

Methodology

Methodology

Executive Summary

- Data collection methodology:
 - Data Collection: Used SpaceX REST API & public sources
- Perform data wrangling
 - Cleaned payloads, encoded categorical variables, handled missing values
- Perform exploratory data analysis (EDA) using visualization and SQL
- Perform interactive visual analytics using Folium and Plotly Dash
- Perform predictive analysis using classification models
 - How to build, tune, evaluate classification models

Data Collection

Data collection involves gathering and measuring information on specific variables within a defined system, enabling us to answer key questions and evaluate outcomes. In this project, the dataset was obtained through **REST API calls** and **web scraping** from Wikipedia.

For the **REST API**, we initiated a GET request, decoded the JSON response, and converted it into a Pandas DataFrame using `json_normalize()`. The data was then cleaned, missing values were identified, and appropriate values were filled as needed.

For **web scraping**, we used **BeautifulSoup** to extract launch records from HTML tables on Wikipedia, parsed the data, and transformed it into a Pandas DataFrame for further analysis.

Data Collection – SpaceX API

Make a get request using
spacex_url

Using json_normalize
function to parse the result
into Data Frame

Perform Data Wrangling
operations e.g. cleaning etc

- <https://github.com/a6dulrauf/IBM-Applied-Data-Science-Capstone-Project/blob/main/%5B1%5D%20Data%20Collection%20API.ipynb>

```
spacex_url="https://api.spacexdata.com/v4/launches/past"
✓ 0.0s Python

response = requests.get(spacex_url)
✓ 1.6s Python

# Use json_normalize meethod to convert the json result into a dataframe
data = pd.json_normalize(response.json())
✓ 0.0s Python

Using the dataframe data print the first 5 rows

# Get the head of the dataframe
data.head()
✓ 0.0s Python

# Lets take a subset of our dataframe keeping only the features we want and the flight number, and date_utc.
data = data[['rocket', 'payloads', 'launchpad', 'cores', 'flight_number', 'date_utc']]

# We will remove rows with multiple cores because those are falcon rockets with 2 extra rocket boosters and rows that have multi
data = data[data['cores'].map(len)==1]
data = data[data['payloads'].map(len)==1]

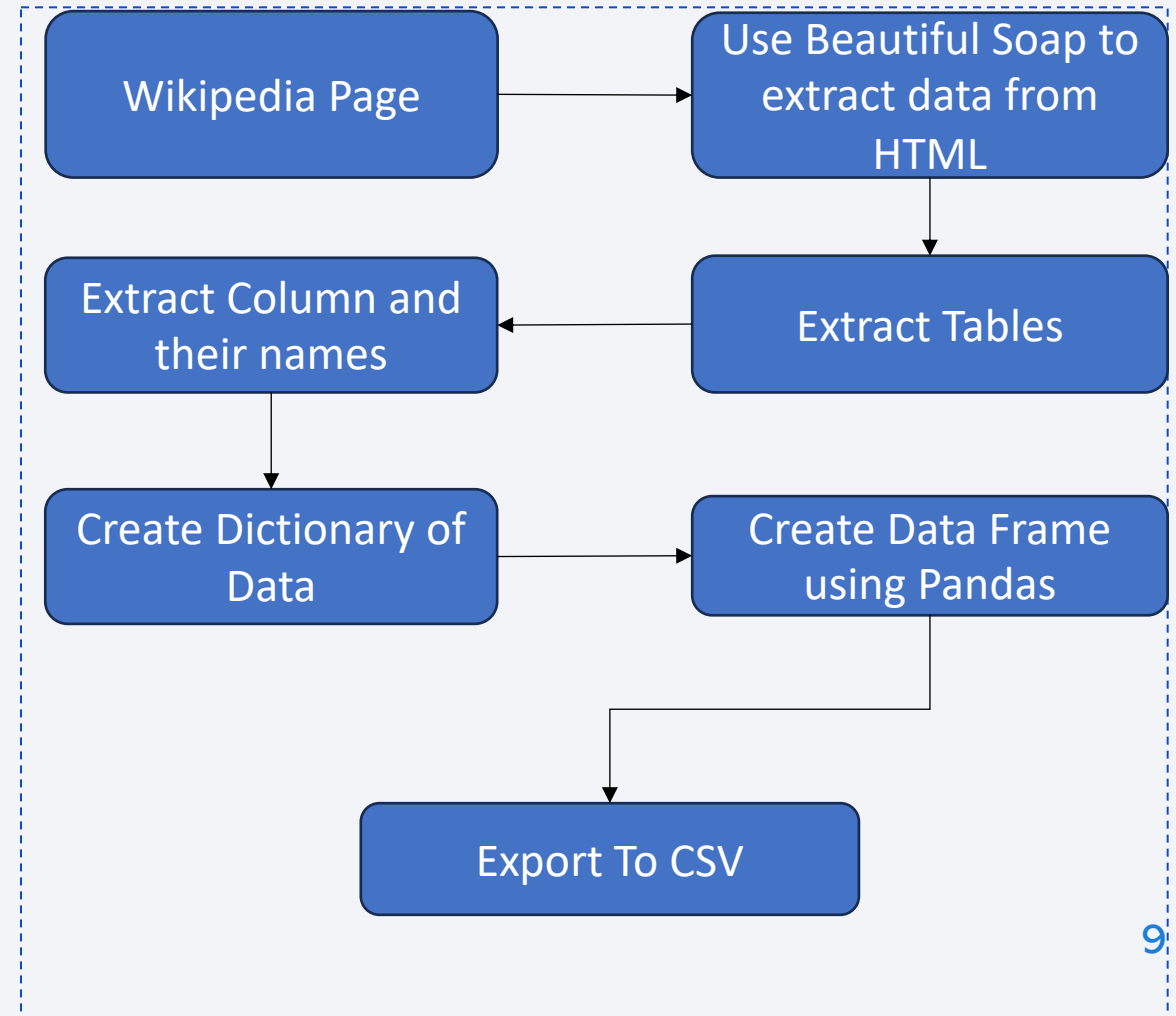
# Since payloads and cores are lists of size 1 we will also extract the single value in the list and replace the feature.
data['cores'] = data['cores'].map(lambda x : x[0])
data['payloads'] = data['payloads'].map(lambda x : x[0])

# We also want to convert the date_utc to a datetime datatype and then extracting the date leaving the time
data['date'] = pd.to_datetime(data['date_utc']).dt.date

# Using the date we will restrict the dates of the launches
data = data[data['date'] <= datetime.date(2020, 11, 13)]
✓ 0.0s Python
```

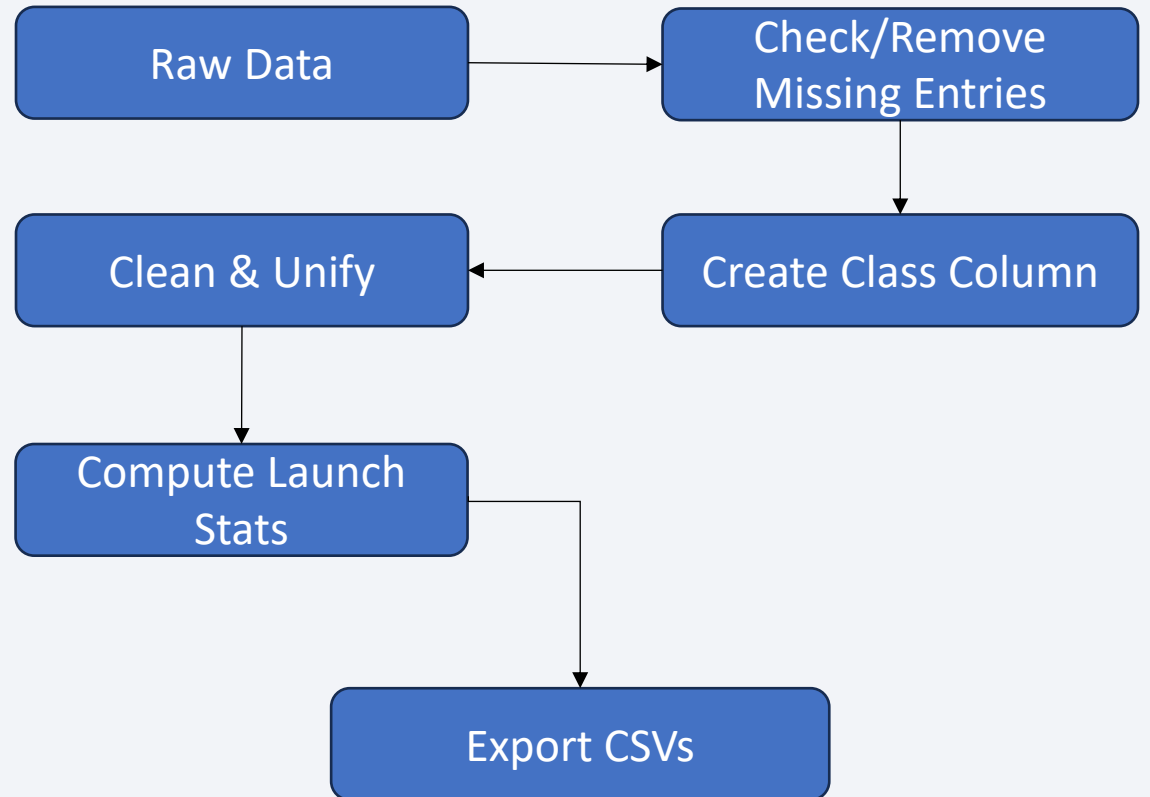

Data Collection - Scraping

- Scraped Falcon 9 launch data from [Wikipedia](#)
- Extracted **121 rows** (launches) and **10 columns** (features)
- Extract out the column names and create an empty dictionary
- Fill out the dictionary using loops
- Create a Data Frame using pandas and export it to a CSV file.
- <https://github.com/a6dulrauf/IBM-Applied-Data-Science-Capstone-Project/blob/main/%5B2%5D%20Data%20Collection%20with%20Web%20Scraping.ipynb>



Data Wrangling

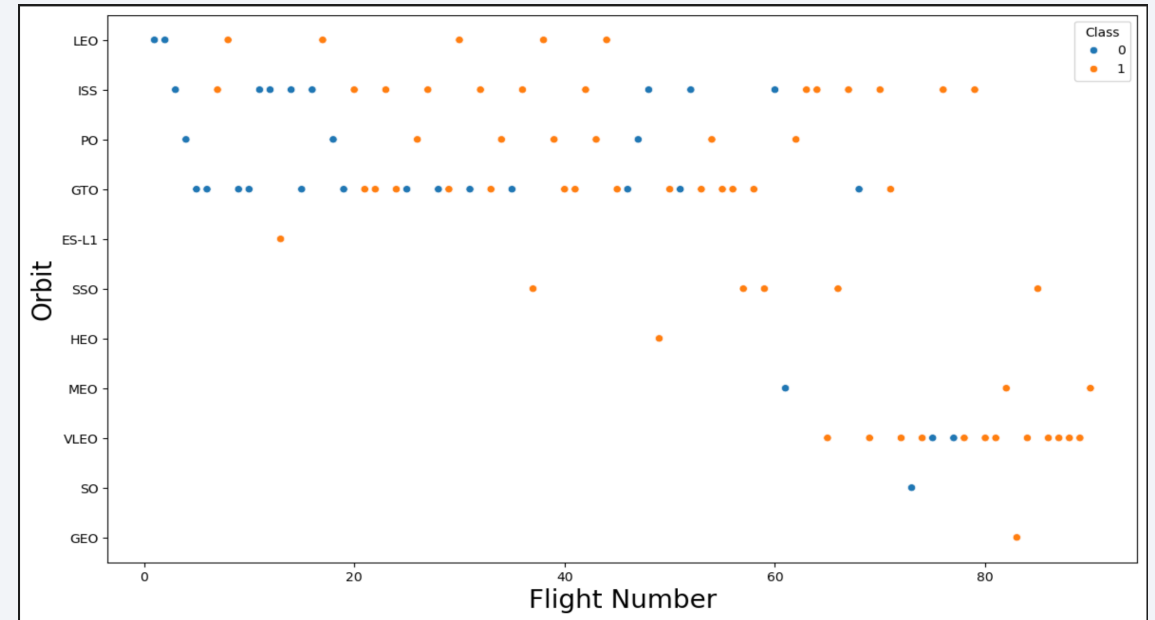
- Processed dataset to remove **missing entries** and ensure consistency
- Created a landing outcome label, namely **'Class'**, from the outcome column
- Column 'Class' contains 0 if a given launch is failed and 1 if it is successful
- Cleaned and unified dataset for **EDA** and **ML tasks**
- Computed launches per site & mission outcomes per orbit type
- Exported the processed dataset to CSV for modeling
- <https://github.com/a6dulrauf/IBM-Applied-Data-Science-Capstone-Project/blob/main/%5B3%5D%20Data%20Wrangling.ipynb>



EDA with Data Visualization

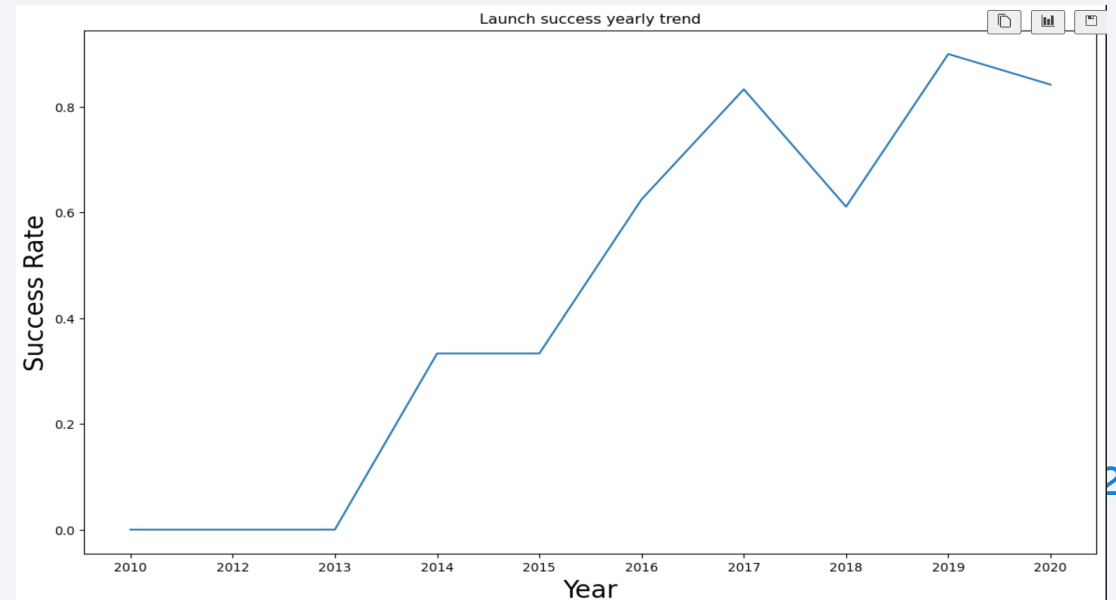
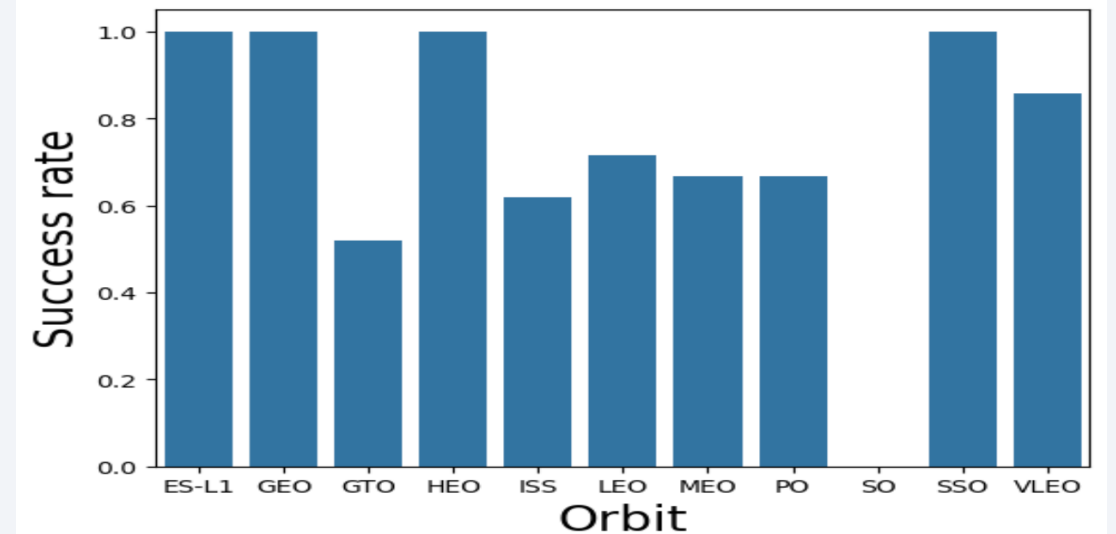
We began our analysis with **scatter plots** to explore the relationships between different launch attributes, such as:

- Payload Mass (kg) vs Flight Number
- Flight Number vs Launch Site
- Payload Mass (kg) vs Launch Site
- Flight Number vs Orbit Type
- Payload vs Orbit Type
- These scatter plots helped us identify dependencies between attributes and observe patterns that could influence the landing outcome. Once patterns were visible, it became easier to determine which factors have the strongest impact on Falcon 9 first-stage landing success.
- <https://github.com/a6dulrauf/IBM-Applied-Data-Science-Capstone-Project/blob/main/%5B6%5D%20EDA%20Visualization.ipynb>



EDA with Data Visualization

- Orbit type vs success rate (bar chart)
 - ES-L1, SSO, HEO, and GEO → **100% success rate**
 - Other orbits → **greater variation in outcomes**
- Yearly trend in success rates (line chart)
 - Early years → **low success**
 - Gradual improvement over time
 - After 2018 → **near 100% reliability**
- <https://github.com/a6dulrauf/IBM-Applied-Data-Science-Capstone-Project/blob/main/%5B6%5D%20EDA%20Visualization.ipynb>



EDA with SQL

Using **SQL queries**, we explored the dataset to gain deeper insights. Examples include:

- Retrieving the **list of launch site names**
- Displaying **five records** where launch sites start with "CCA"
- Calculating the **total payload mass** for boosters launched under NASA (CRS) missions
- Finding the **average payload mass** carried by booster version **F9 v1.1**
- Identifying the **date of the first successful ground pad landing**
- Listing **boosters with successful drone ship landings** carrying payloads between **4000–6000 kg**
- Counting the **number of successful vs. failed mission outcomes**
- Determining which **booster versions carried the maximum payload mass**
- Extracting **failed drone ship landings in 2015**, including booster versions and launch sites
- Ranking the **frequency of landing outcomes** (success/failure) between **2010-06-04 and 2017-03-20**, in descending order
- <https://github.com/a6dulrauf/IBM-Applied-Data-Science-Capstone-Project/blob/main/%5B4%5D%20EDA%20with%20SQL.ipynb>

Build an Interactive Map with Folium

To create an **interactive map visualization** of the launch data:

- Used **latitude and longitude** coordinates to plot each launch site with **circle markers** labeled by site name
- Assigned launch outcomes as classes:
 - **Class 1 (Success)** → **Green markers**
 - **Class 0 (Failure)** → **Red markers**, grouped with **MarkerCluster()**
- Applied the **Haversine formula** to calculate distances from launch sites to landmarks, answering questions such as:
 - How close are launch sites to **railways, highways, and coastlines**?
 - How close are launch sites to **nearby cities**?
- <https://github.com/a6dulrauf/IBM-Applied-Data-Science-Capstone-Project/blob/main/%5B7%5D%20Interactive%20Visual%20Analytics%20with%20Folium%20lab.ipynb>

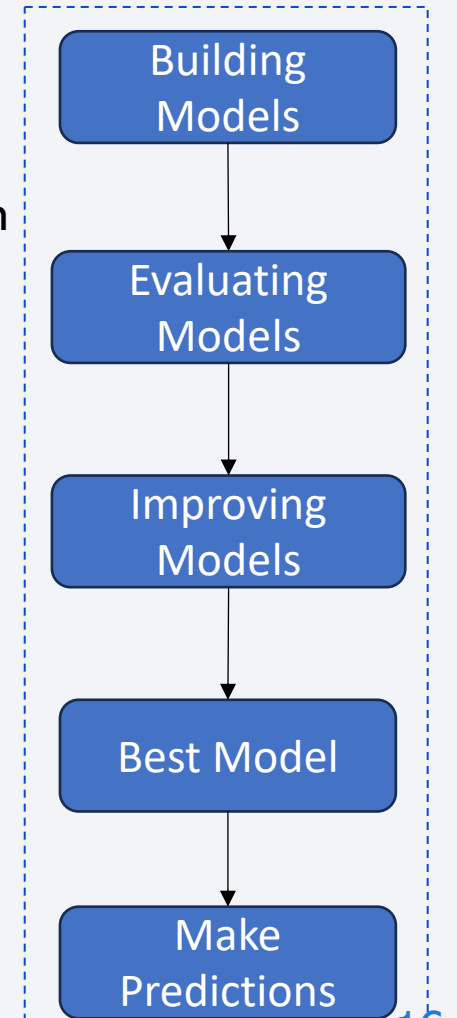
Build a Dashboard with Plotly Dash

We developed an **interactive dashboard** using **Plotly Dash** to allow users to explore the data dynamically.

- Added **pie charts** to display the distribution of total launches by site
- Created a **scatter plot** to visualize the relationship between **payload mass (kg)** and **landing outcome** across different **booster versions**
- <https://github.com/a6dulrauf/IBM-Applied-Data-Science-Capstone-Project/blob/main/%5B8%5D%20spacex%20dash%20app.py>

Predictive Analysis (Classification)

- **Building the Models**
 - Standardized and transformed the dataset
 - Split data into **training** and **test sets**
 - Implemented **Logistic Regression, SVM, Decision Tree, KNN** using Scikit-learn
 - Used **GridSearchCV** for hyperparameter tuning
- **Evaluating the Models**
 - Measured **accuracy scores** on the test set
 - Analyzed **confusion matrices** for success/failure predictions
 - Compared tuned hyperparameters across models
- **Improving the Models**
 - Applied **feature engineering**
 - Refined parameters with **algorithm tuning**
- **Selecting the Best Model**
 - **Decision Tree** achieved the highest accuracy (**0.886**)
 - Chosen as the **best-performing model**
- <https://github.com/a6dulrauf/IBM-Applied-Data-Science-Capstone-Project/blob/main/%5B9%5D%20Machine%20Learning%20Prediction.ipynb>



Results

Our results are organized into three main categories:

- **Exploratory Data Analysis (EDA)**

- Conducted using **SQL queries**, **Matplotlib/Seaborn visualizations**, and **Folium maps**
- Showed patterns in launch outcomes, payloads, orbits, and sites

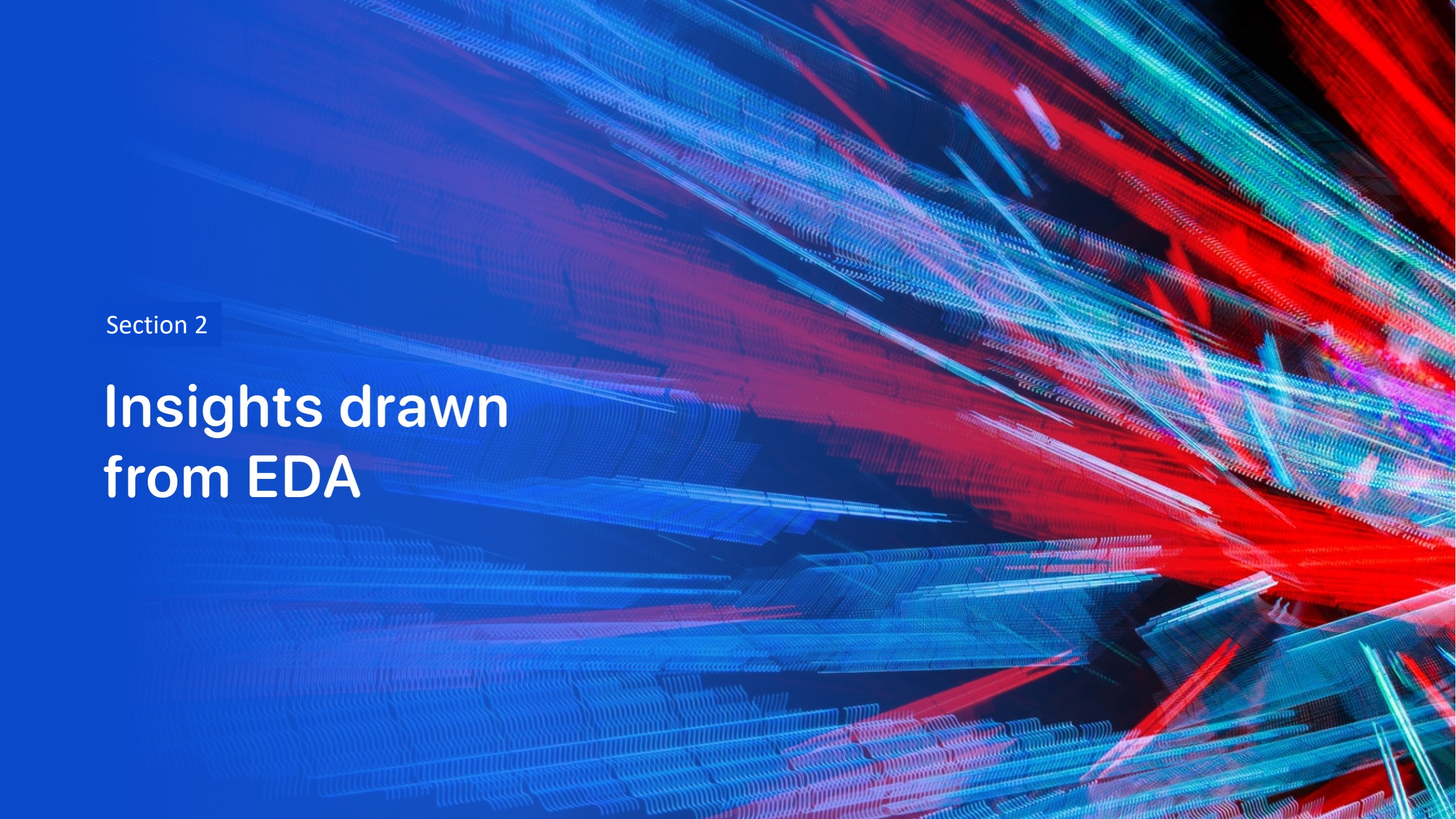
- **Interactive Analytics**

- Built with **Plotly Dash**, providing dynamic exploration through **pie charts**, **scatter plots**, and **filters**
- Folium maps visualized launch site locations and distances to landmarks

- **Predictive Analysis**

- Applied multiple machine learning algorithms (Logistic Regression, SVM, KNN, Decision Tree)
- **Decision Tree** achieved the highest accuracy (**0.886**)

In all graphs and models, **Class 0 = failed launch** and **Class 1 = successful launch**.

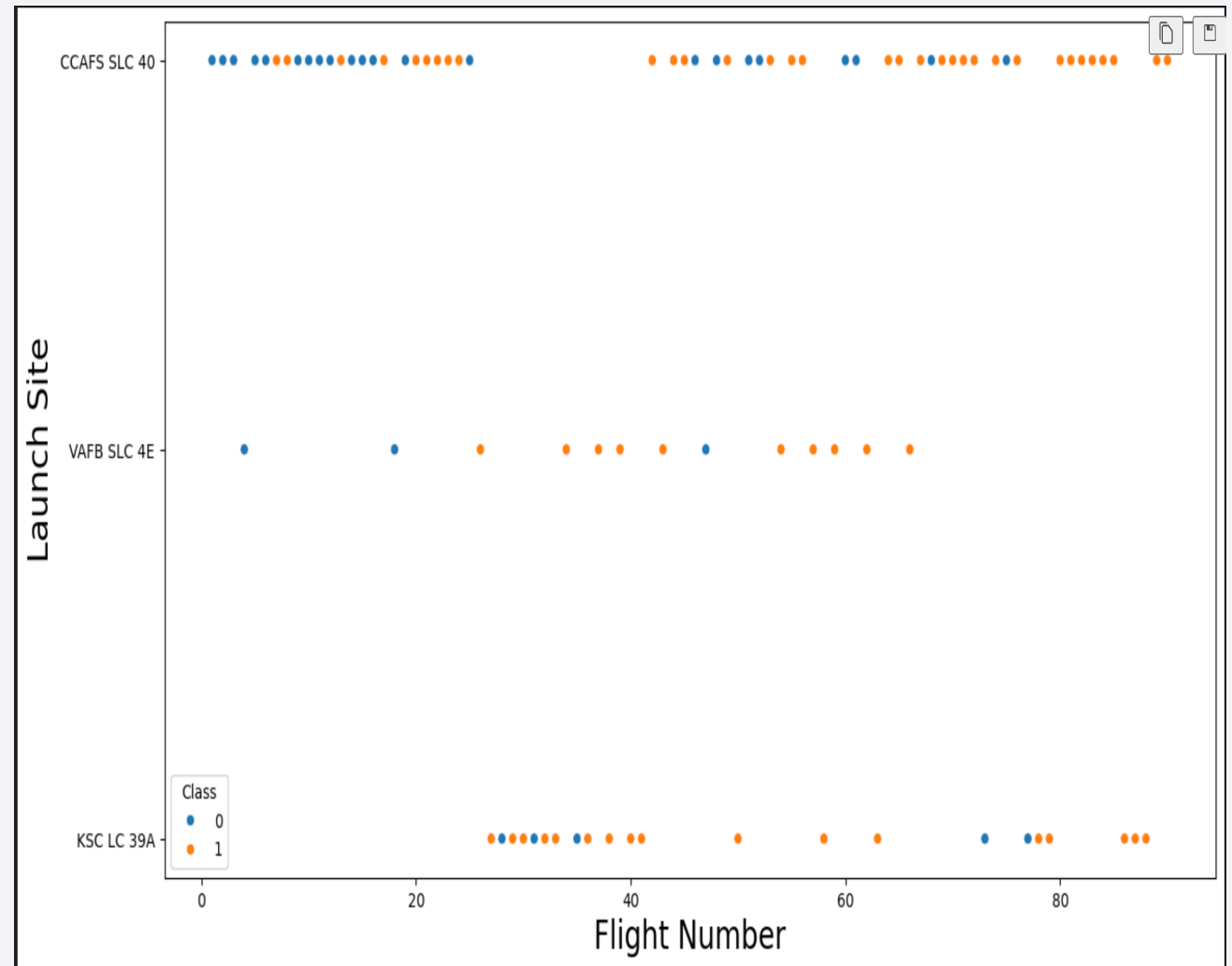
The background of the slide is an abstract composition. It features a dark blue base color. Overlaid on this are numerous diagonal streaks in shades of red and cyan. A faint, light blue grid pattern is also visible, particularly in the lower-left quadrant. The overall effect is dynamic and technological.

Section 2

Insights drawn from EDA

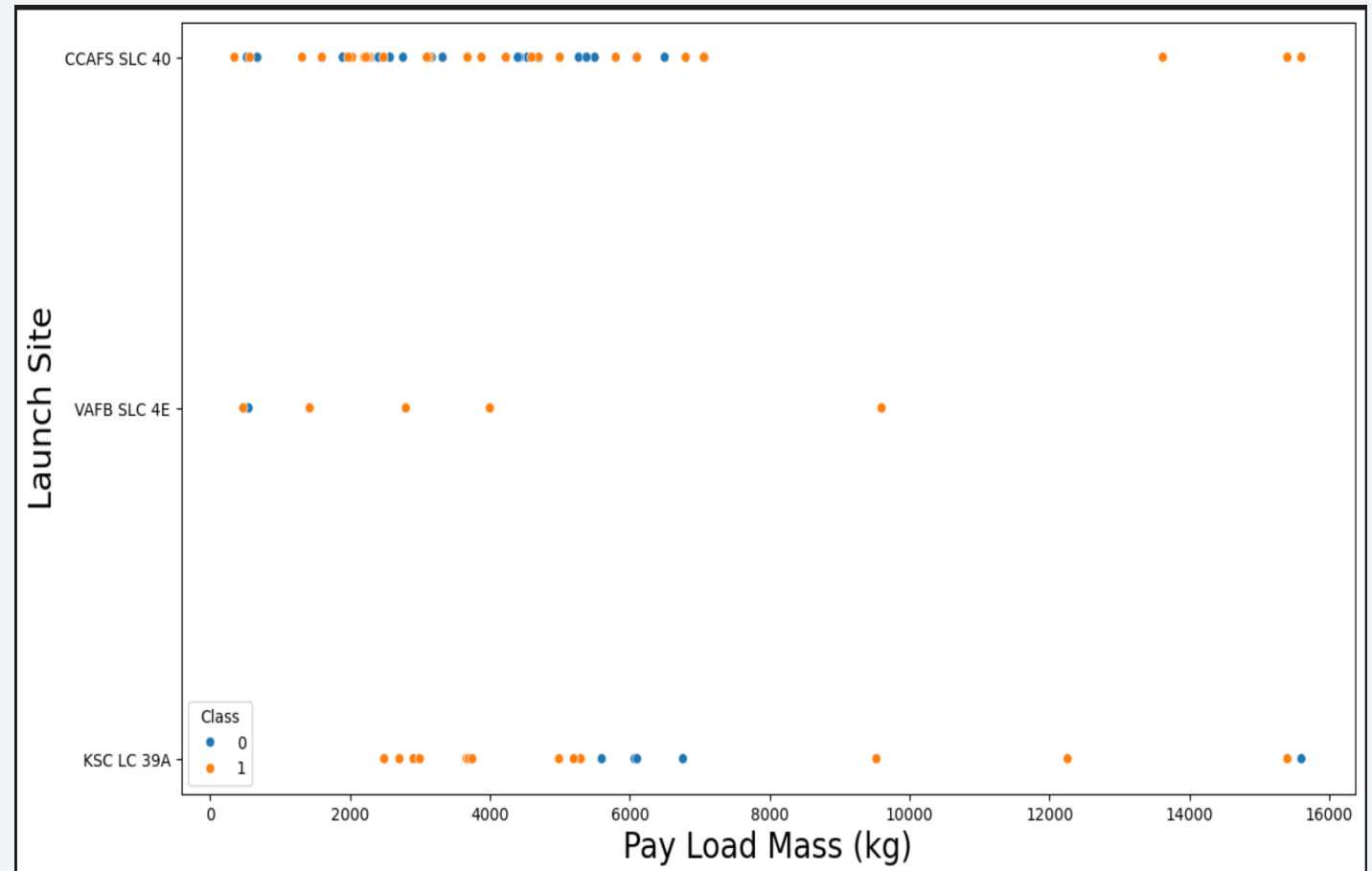
Flight Number vs. Launch Site

- This scatter plot shows that **launch sites with more flights tend to have higher success rates**. However, the site **CCAFS SLC40** does not strongly follow this pattern, showing a weaker correlation between flight count and success rate.



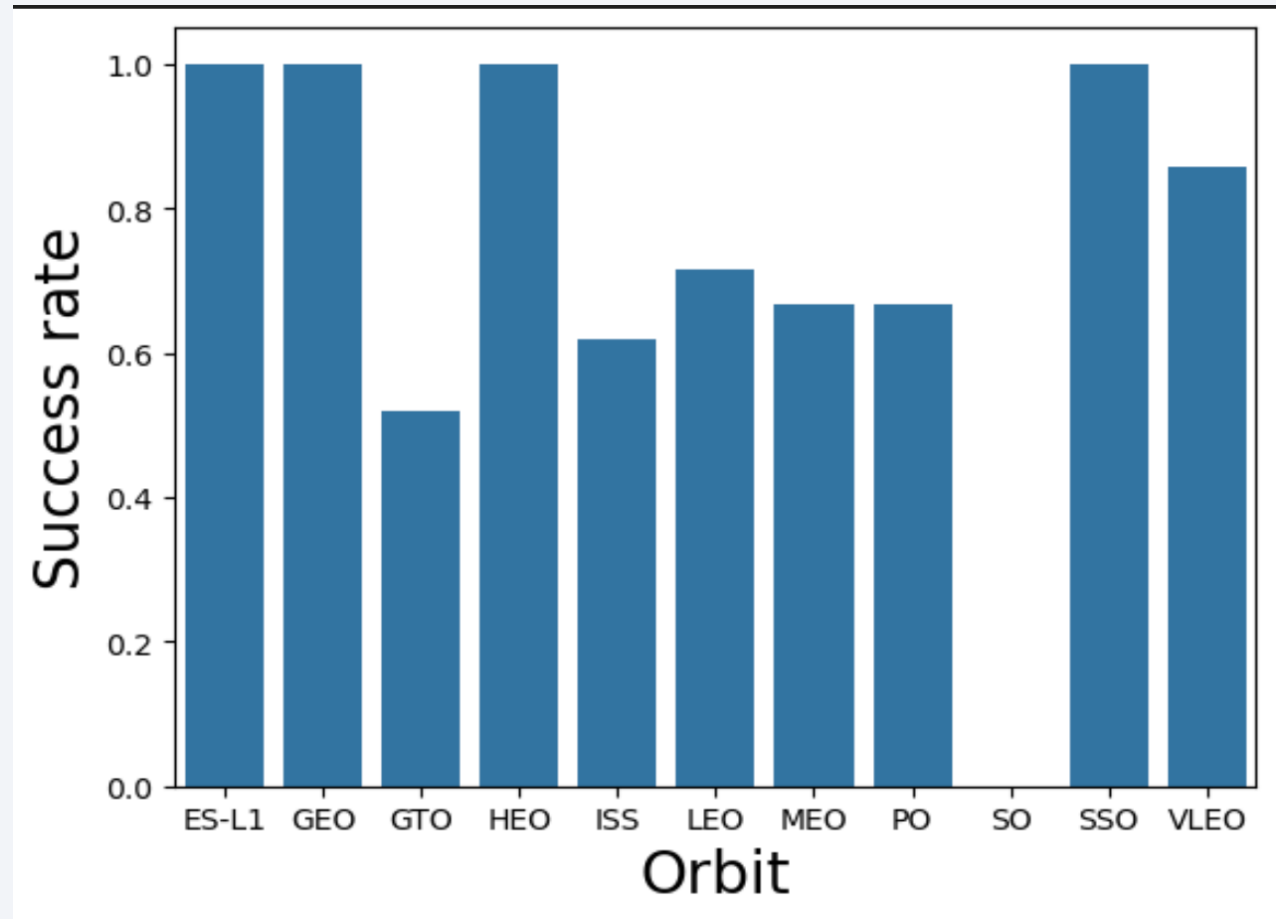
Payload vs. Launch Site

- This scatter plot indicates that **payloads above 7000 kg are more likely to result in successful launches**. However, there is **no clear evidence that launch site success is directly dependent on payload mass**.



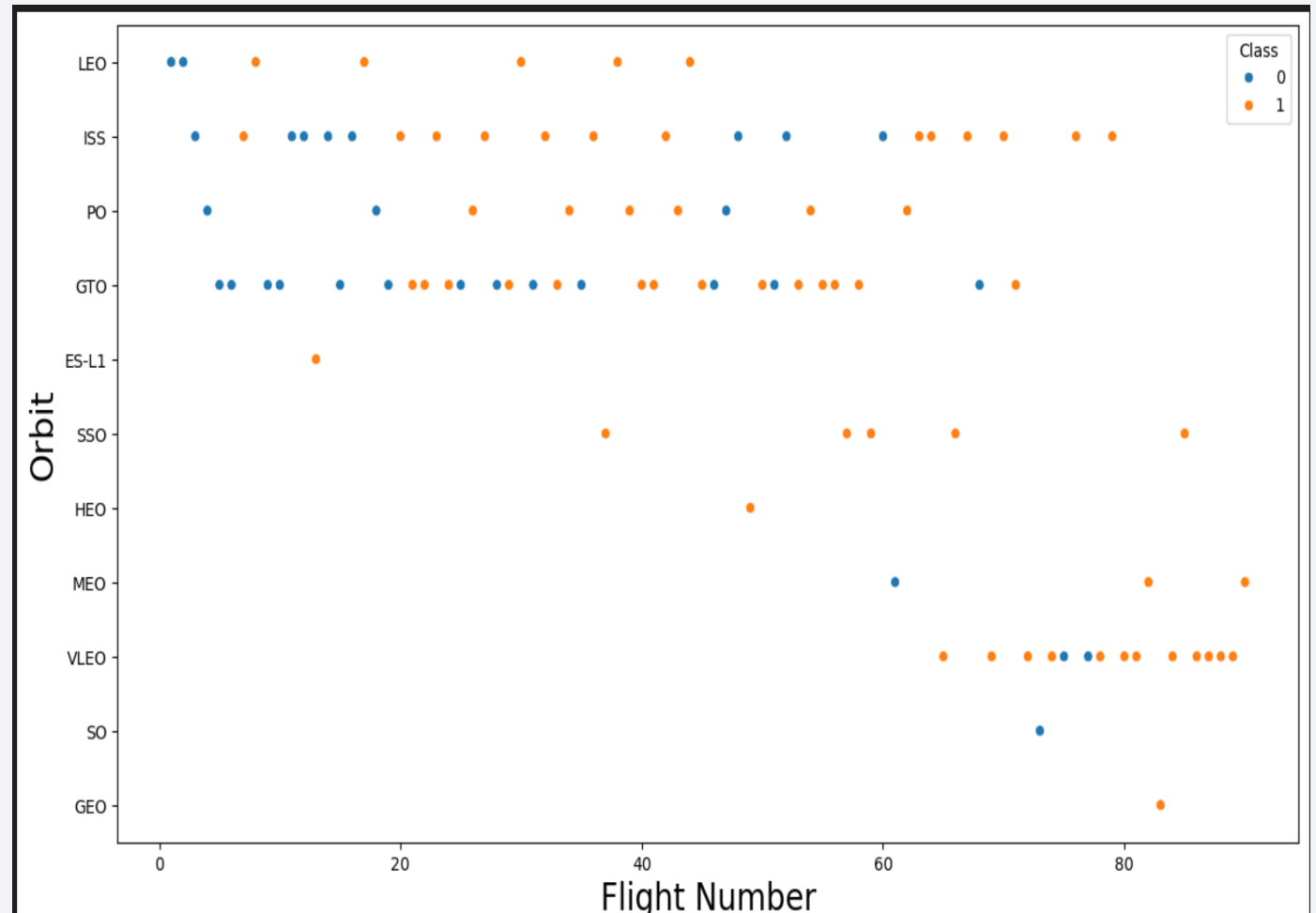
Success Rate vs. Orbit Type

- This figure shows that **certain orbits appear to influence landing outcomes**. For example, **SSO, HEO, GEO, and ES-L1** achieved a **100% success rate**, while **SO orbit had 0% success**. However, deeper analysis reveals that some of these orbits (e.g., **GEO, SO, HEO, ES-L1**) had only a **single occurrence**, meaning more data is needed before drawing reliable conclusions.



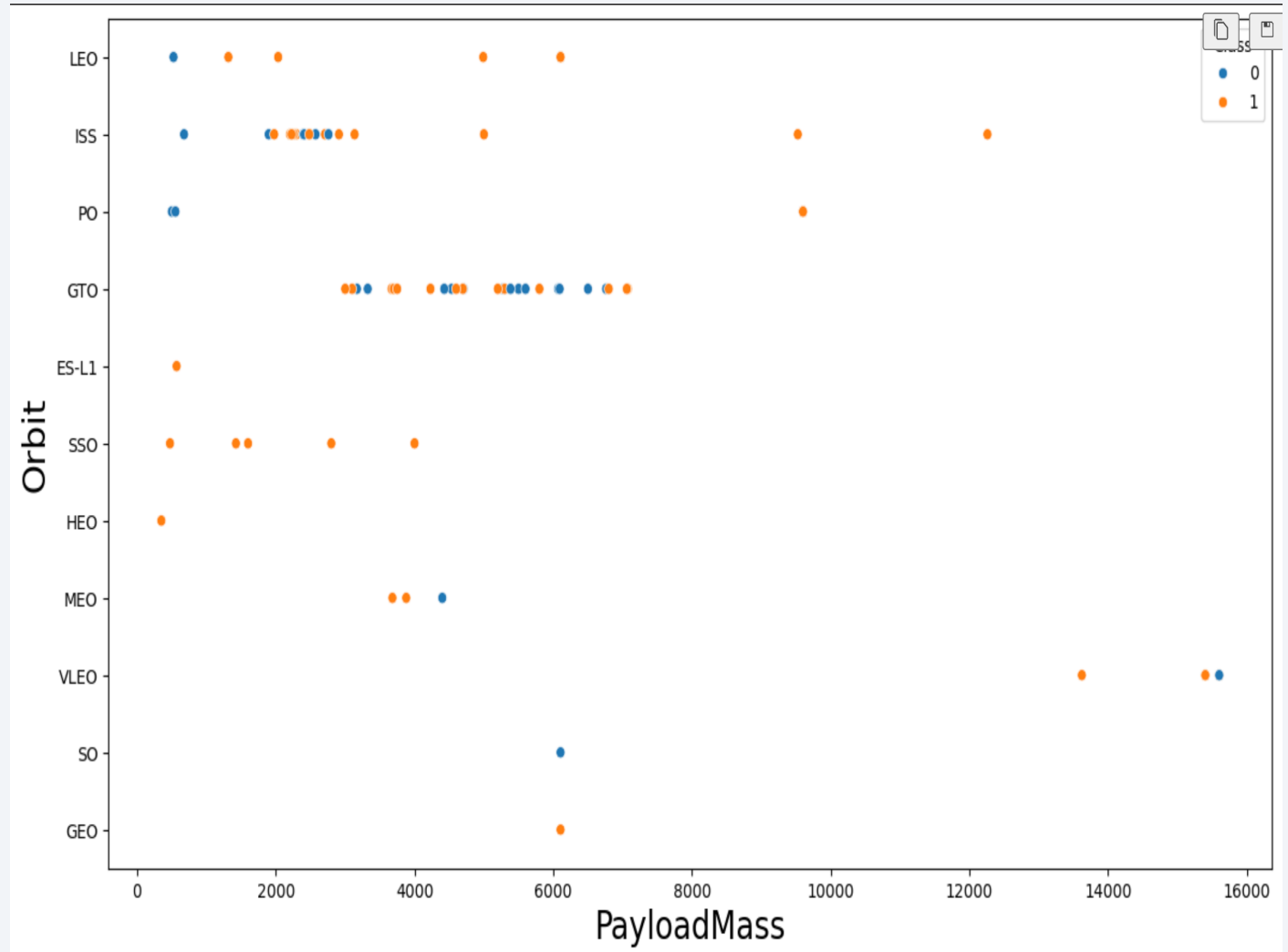
Flight Number vs. Orbit Type

- This scatter plot suggests that **higher flight numbers generally correlate with greater success rates**, particularly for the **LEO** orbit. However, the **GTO** orbit shows **no clear relationship** between flight number and success. Orbits with only a **single occurrence** should be excluded from this observation, as more data is required to establish patterns.



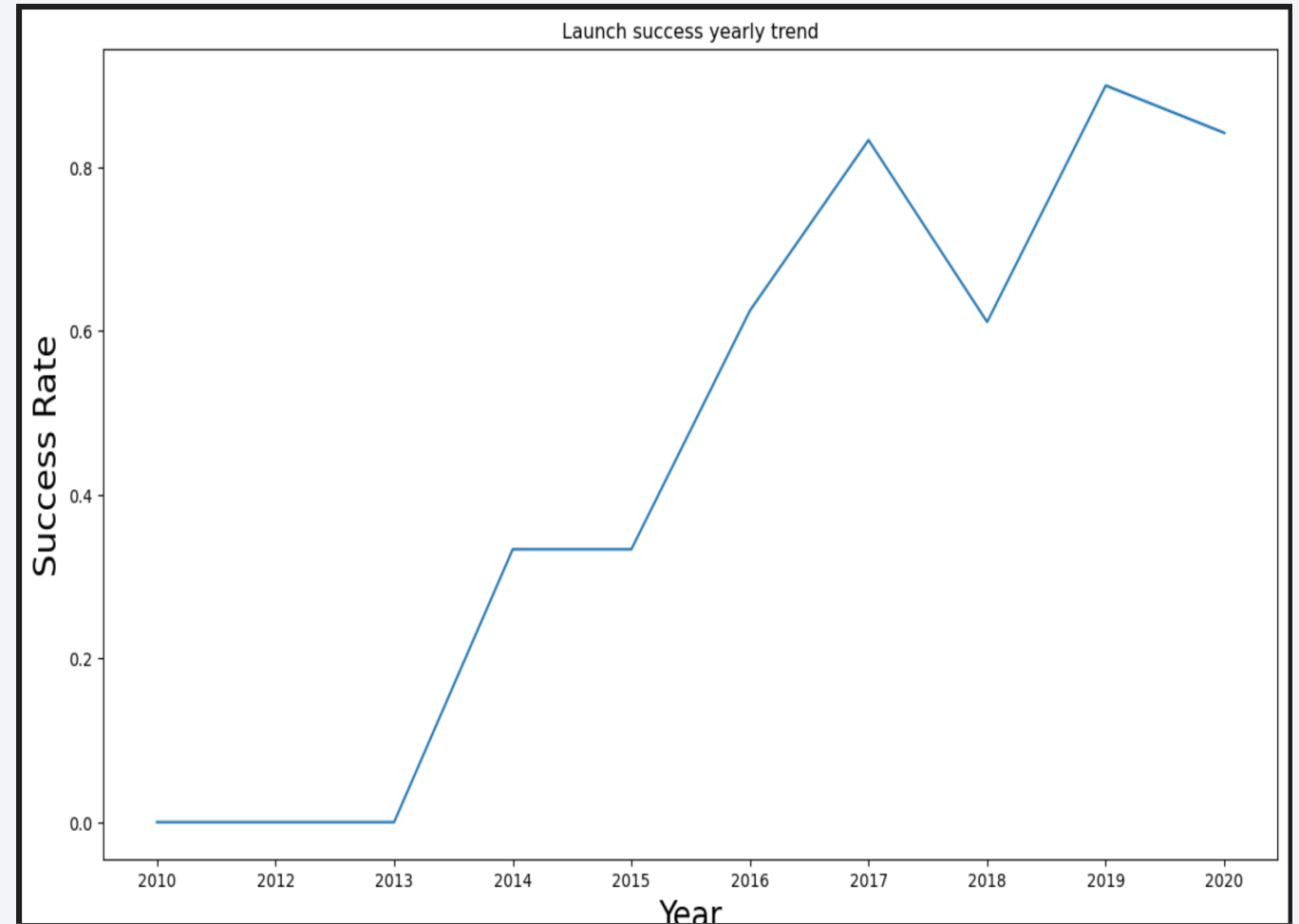
Payload vs. Orbit Type

- Heavier payloads show a **positive impact on success rates for LEO, ISS, and PO orbits**, but a **negative impact for MEO and VLEO orbits**. For **GTO orbits**, no clear relationship is observed. As with earlier cases, orbits such as **SO, GEO, and HEO** had very few launches, so more data is required to identify meaningful trends.



Launch Success Yearly Trend

- These figures clearly show an **increasing trend in success rates from 2013 to 2020**. If this trend continues, SpaceX launch success rates are expected to **approach 100% reliability in the coming years**.



All Launch Site Names

- We used the SQL keyword **DISTINCT** to retrieve only the **unique launch sites** from the SpaceX dataset.

```
> %sql SELECT DISTINCT LAUNCH_SITE as "Launch_Sites" FROM SPACEXTBL;
[16]
* ibm_db_sa://dgy37633:***@98538591-7217-4024-b027-8baa776ffad1.c3n41cmd0nqnrk39u98g.databases.appdomain.cloud:30875/bludb
Done.

Launch_Sites
CCAFS LC-40
CCAFS SLC-40
KSC LC-39A
VAFB SLC-4E
```

Launch Site Names Begin with 'CCA'

- We used a SQL query with the **LIKE 'CCA%'** condition to display **five records** where the launch site name begins with **CCA**.

```
%sql SELECT * FROM SPACEXTBL WHERE LAUNCH_SITE LIKE 'CCA%' LIMIT 5;
```

Python

```
* ibm_db_sa://dgy37633:***@98538591-7217-4024-b027-8baa776ffad1.c3n41cmd0nqnrk39u98g.databases.appdomain.cloud:30875/bludb
Done.
```

DATE	time__utc__	booster_version	launch_site	payload	payload_mass__kg__	orbit	customer	mission_outcome	landing__outcome
2010-06-04	18:45:00	F9 v1.0 B0003	CCAFS LC-40	Dragon Spacecraft Qualification Unit	0	LEO	SpaceX	Success	Failure (parachute)
2010-12-08	15:43:00	F9 v1.0 B0004	CCAFS LC-40	Dragon demo flight C1, two CubeSats, barrel of Brouere cheese	0	LEO (ISS)	NASA (COTS) NRO	Success	Failure (parachute)
2012-05-22	07:44:00	F9 v1.0 B0005	CCAFS LC-40	Dragon demo flight C2	525	LEO (ISS)	NASA (COTS)	Success	No attempt
2012-10-08	00:35:00	F9 v1.0 B0006	CCAFS LC-40	SpaceX CRS-1	500	LEO (ISS)	NASA (CRS)	Success	No attempt
2013-03-01	15:10:00	F9 v1.0 B0007	CCAFS LC-40	SpaceX CRS-2	677	LEO (ISS)	NASA (CRS)	Success	No attempt

Generate

+ Code

+ Markdown

Total Payload Mass

- We used an SQL query to calculate the **total payload mass** carried by boosters from **NASA (CRS)**, which amounted to **45,596 kg**.

```
%sql SELECT SUM(PAYLOAD_MASS_KG_) AS "Total payload mass by NASA (CRS)" FROM SPACEXTBL WHERE CUSTOMER = 'NASA (CRS)';
```

Python

```
* ibm_db_sa://dgy37633:***@98538591-7217-4024-b027-8baa776ffad1.c3n41cmd0nqnrk39u98g.databases.appdomain.cloud:30875/bludb  
Done.
```

Total payload mass by NASA (CRS)

45596

Average Payload Mass by F9 v1.1

- We calculated the **average payload mass** carried by booster version **F9 v1.1**, which was **2,928 kg**.

```
%sql SELECT AVG(PAYLOAD_MASS_KG_) AS "Average payload mass by Booster Version F9 v1.1" FROM SPACEXTBL WHERE BOOSTER_VERSION = 'F9 v1.1';
```

Python

```
* ibm_db_sa://dgy37633:***@98538591-7217-4024-b027-8baa776ffad1.c3n41cmd0nqnrk39u98g.databases.appdomain.cloud:30875/bludb  
Done.
```

Average payload mass by Booster Version F9 v1.1

2928

First Successful Ground Landing Date

- We used the SQL `MIN()` function to identify the **earliest successful ground pad landing**, which occurred on **22nd December 2015**.

```
%sql SELECT MIN(DATE) AS "Date of first successful landing outcome in ground pad" FROM SPACEXTBL WHERE LANDING__OUTCOME = 'Success (ground pad)';
```

Python

```
* ibm_db_sa://dgy37633:***@98538591-7217-4024-b027-8baa776ffad1.c3n41cmd0nqnrk39u98g.databases.appdomain.cloud:30875/bludb  
Done.
```

```
Date of first successful landing outcome in ground pad
```

```
2015-12-22
```

Successful Drone Ship Landing with Payload between 4000 and 6000

- We used the SQL **WHERE** clause to filter boosters with **successful drone ship landings**, and applied the **AND** condition to select those carrying a **payload mass between 4000 and 6000 kg**.

```
%sql SELECT BOOSTER_VERSION FROM SPACEXTBL WHERE LANDING__OUTCOME = 'Success (drone ship)' AND PAYLOAD_MASS__KG_ BETWEEN 4000 AND 6000;
```

Python

```
* ibm_db_sa://dgy37633:***@98538591-7217-4024-b027-8baa776ffad1.c3n41cmd0nqnrk39u98g.databases.appdomain.cloud:30875/bludb  
Done.
```

```
booster_version
```

```
F9 FT B1022
```

```
F9 FT B1026
```

```
F9 FT B1021.2
```

```
F9 FT B1031.2
```

Total Number of Successful and Failure Mission Outcomes

- We used the SQL wildcard % with the LIKE operator to filter records where Mission_Outcome indicated either a success or a failure.

```
%sql SELECT number_of_success_outcomes, number_of_failure_outcomes FROM (SELECT COUNT(*) AS number_of_success_outcomes FROM SPACEXTBL WHERE MISSION_OUTCO
```

Python

```
[ * ibm_db_sa://dgy37633:***@98538591-7217-4024-b027-8baa776ffad1.c3n41cmd0nqn timerk39u98g.databases.appdomain.cloud:30875/bludb  
Done.
```

number_of_success_outcomes	number_of_failure_outcomes
100	1

Boosters Carried Maximum Payload

- We used the SQL MAX() function along with a subquery in the WHERE clause to identify the booster that carried the maximum payload mass.

```
%sql SELECT DISTINCT BOOSTER_VERSION FROM SPACEXTBL WHERE PAYLOAD_MASS_KG_ =(SELECT MAX(PAYLOAD_MASS_KG_) FROM SPACEXTBL);
```

Python

```
* ibm_db_sa://dgy37633:***@98538591-7217-4024-b027-8baa776ffad1.c3n41cmd0nqnrk39u98g.databases.appdomain.cloud:30875/bludb  
Done.
```

```
booster_version
```

```
F9 B5 B1048.4
```

```
F9 B5 B1048.5
```

```
F9 B5 B1049.4
```

```
F9 B5 B1049.5
```

```
F9 B5 B1049.7
```

```
F9 B5 B1051.3
```

```
F9 B5 B1051.4
```

```
F9 B5 B1051.6
```

```
F9 B5 B1056.4
```

```
F9 B5 B1058.3
```

```
F9 B5 B1060.2
```

```
F9 B5 B1060.3
```

2015 Launch Records

- We combined SQL conditions — **WHERE**, **LIKE**, **AND**, and **BETWEEN** — to filter for **failed drone ship landings** in **2015**, retrieving the corresponding **booster versions** and **launch site names**.

```
%sql SELECT DATE, BOOSTER_VERSION, LAUNCH_SITE FROM SPACEXTBL WHERE year(DATE) = '2015' AND LANDING__OUTCOME = 'Failure (drone ship)';
```

Python

```
* ibm_db_sa://dgy37633:***@98538591-7217-4024-b027-8baa776ffad1.c3n41cmd0nqnrk39u98g.databases.appdomain.cloud:30875/bludb  
Done.
```

DATE	booster_version	launch_site
2015-01-10	F9 v1.1 B1012	CCAFS LC-40
2015-04-14	F9 v1.1 B1015	CCAFS LC-40

Rank Landing Outcomes Between 2010-06-04 and 2017-03-20

We used SQL to analyze landing outcomes within a specific date range:

- Applied **WHERE** to filter records **between 2010-06-04 and 2017-03-20**
- Used **COUNT()** to calculate the number of each landing outcome
- Grouped results with **GROUP BY Landing_Outcome**
- Ordered the outcomes in **descending order** using **ORDER BY**

```
%sql SELECT LANDING__OUTCOME, COUNT(LANDING__OUTCOME) AS Landing_Count FROM SPACEXTBL WHERE DATE BETWEEN '2010-06-04' AND '2017-03-20' GROUP BY LANDING__
```

Python

```
* ibm_db_sa://dgy37633:***@98538591-7217-4024-b027-8baa776ffad1.c3n41cmd0nqnkrk39u98g.databases.appdomain.cloud:30875/bludb  
Done.
```

landing__outcome	landing_count
No attempt	10
Failure (drone ship)	5
Success (drone ship)	5
Controlled (ocean)	3
Success (ground pad)	3
Failure (parachute)	2
Uncontrolled (ocean)	2
Precluded (drone ship)	1

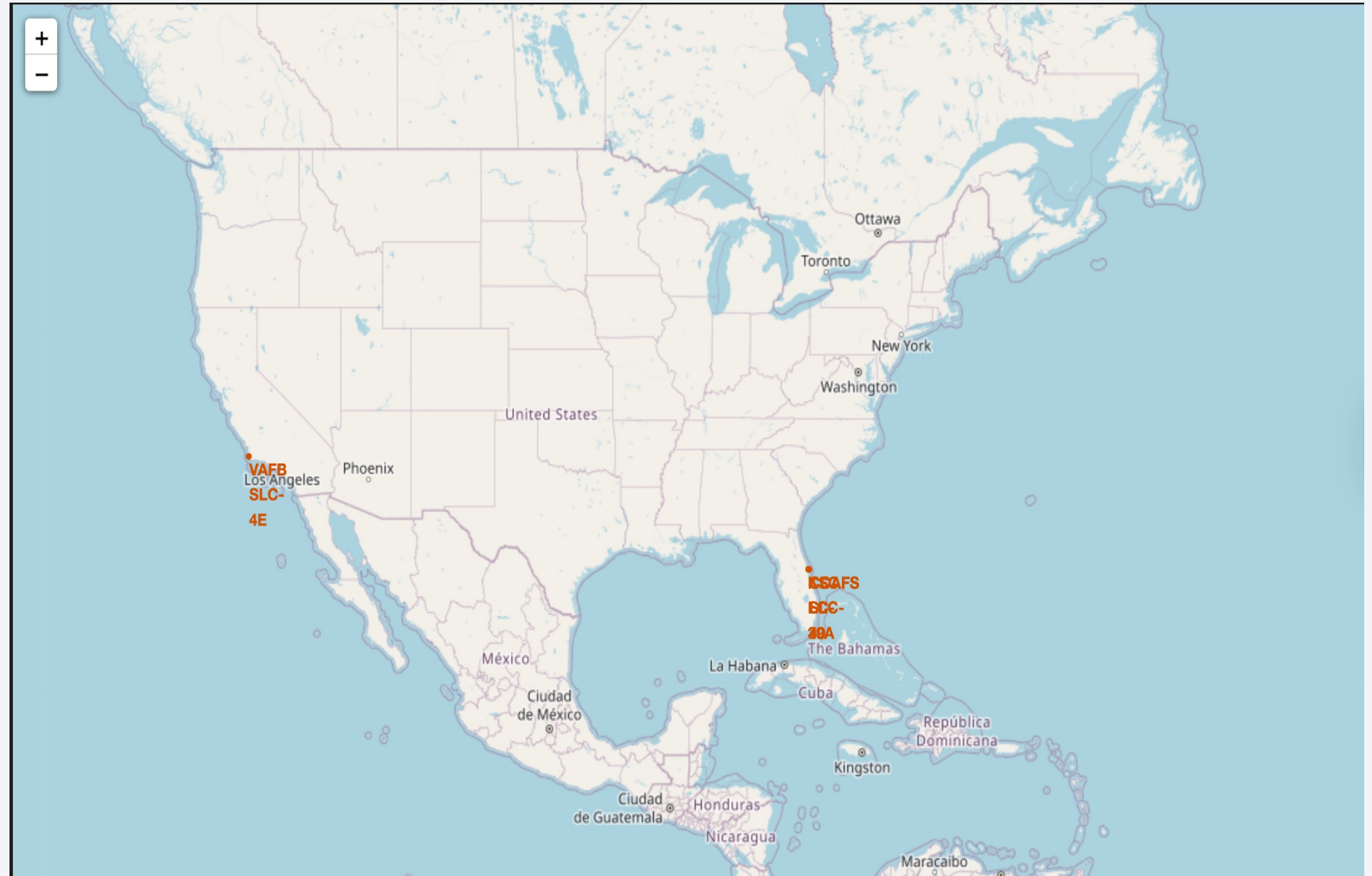
A satellite view of Earth from space, showing the curvature of the planet and city lights at night. The image is a composite of a solid blue area on the left and a satellite photograph of Earth on the right. The Earth's surface is dark, with numerous bright yellow and orange lights representing cities and urban areas. The horizon of the Earth is visible as a curved line separating the dark surface from the deep blue of space.

Section 3

Launch Sites Proximities Analysis

Location of all the Launch Sites

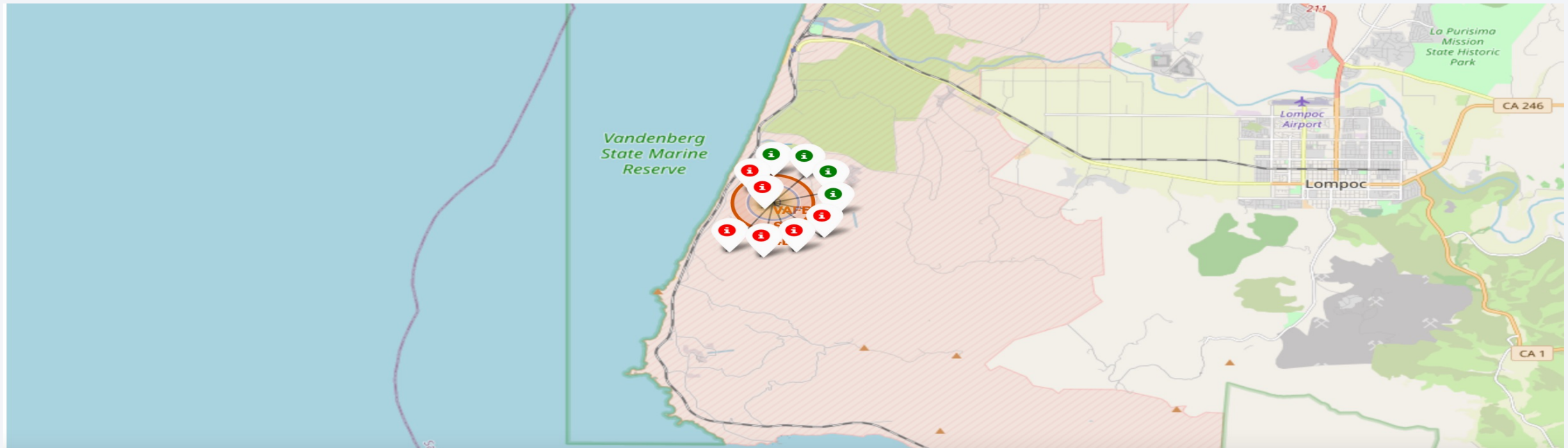
We can see that all the SpaceX Launch sites are located inside the United States.



Markers showing launch sites with color labels

We visualized **launch outcomes** on an **interactive map** for each site.

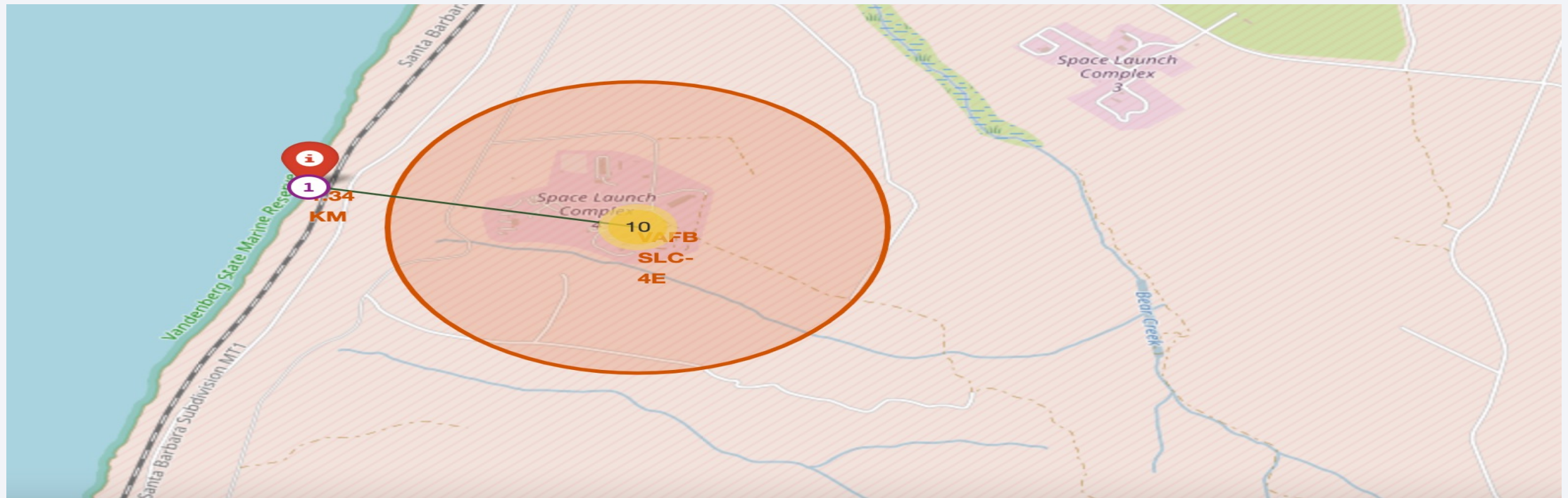
- **Green markers** represent **successful launches**
- **Red markers** represent **failed launches**
- **Zooming into a site** reveals the detailed distribution of outcomes

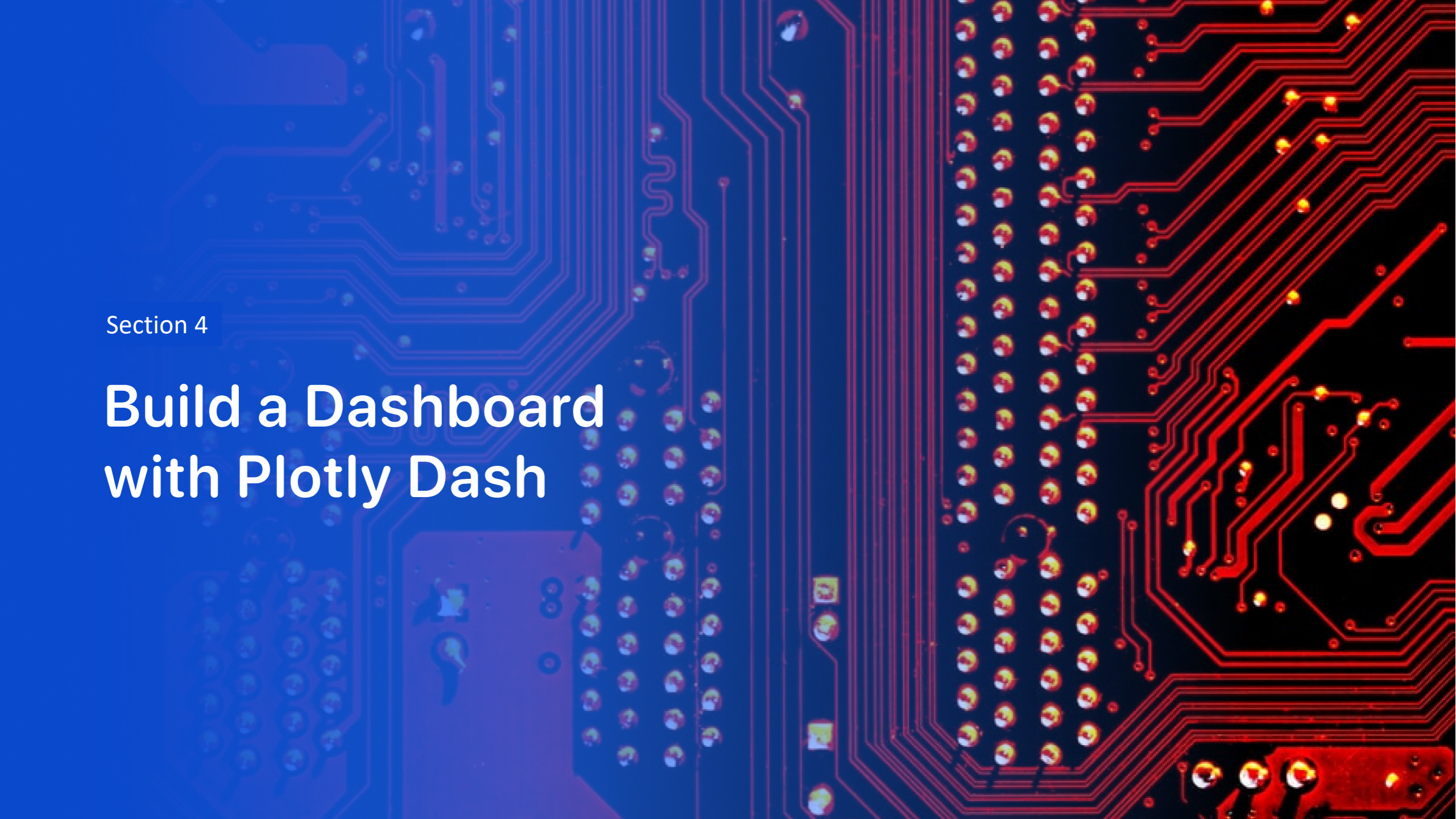


Launch Sites Distance

We analyzed the distances between launch sites and nearby proximities such as the nearest city, railway, highway, and coastline.

- The figure below illustrates the distance from the VAFB SLC-4E launch site to its nearest coastline.





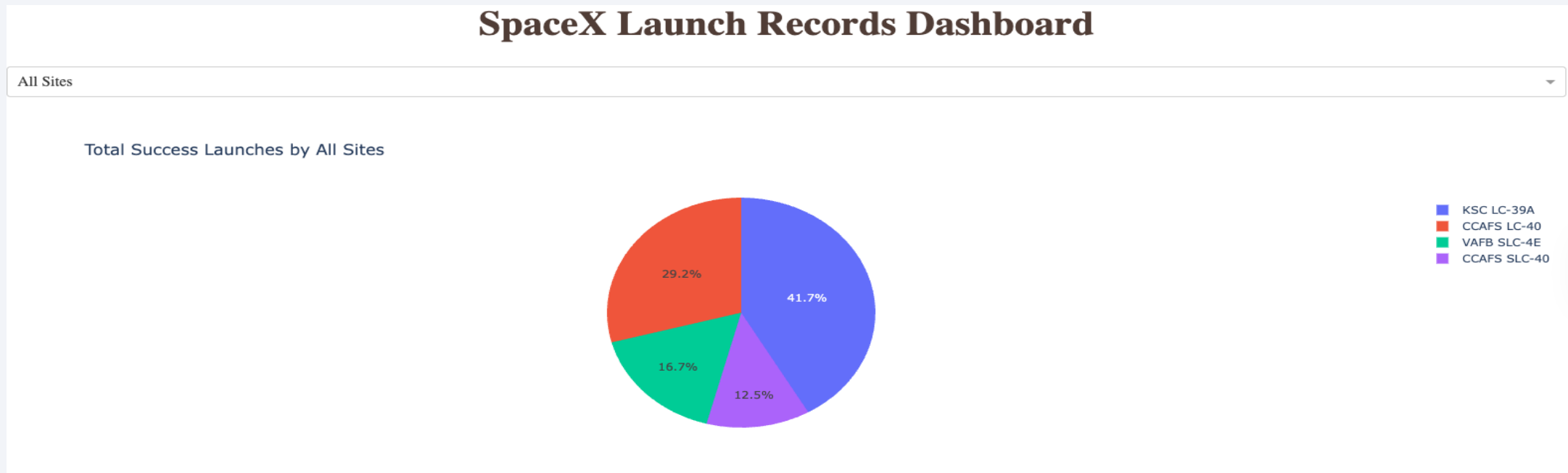
Section 4

Build a Dashboard with Plotly Dash

<Dashboard Screenshot 1>

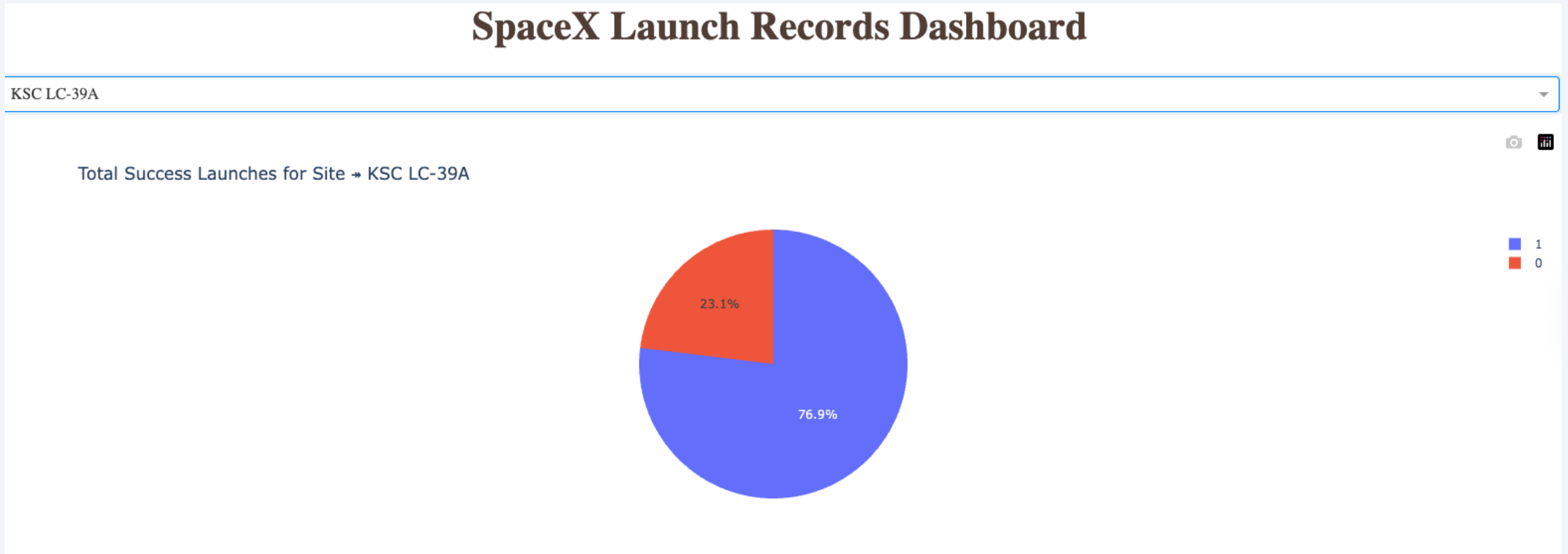
The pie chart below shows the launch outcomes for site CCAFS LC-40:

- Class 0 = Failed launches
- Class 1 = Successful launches
- At this site, 73.1% of launches resulted in failure



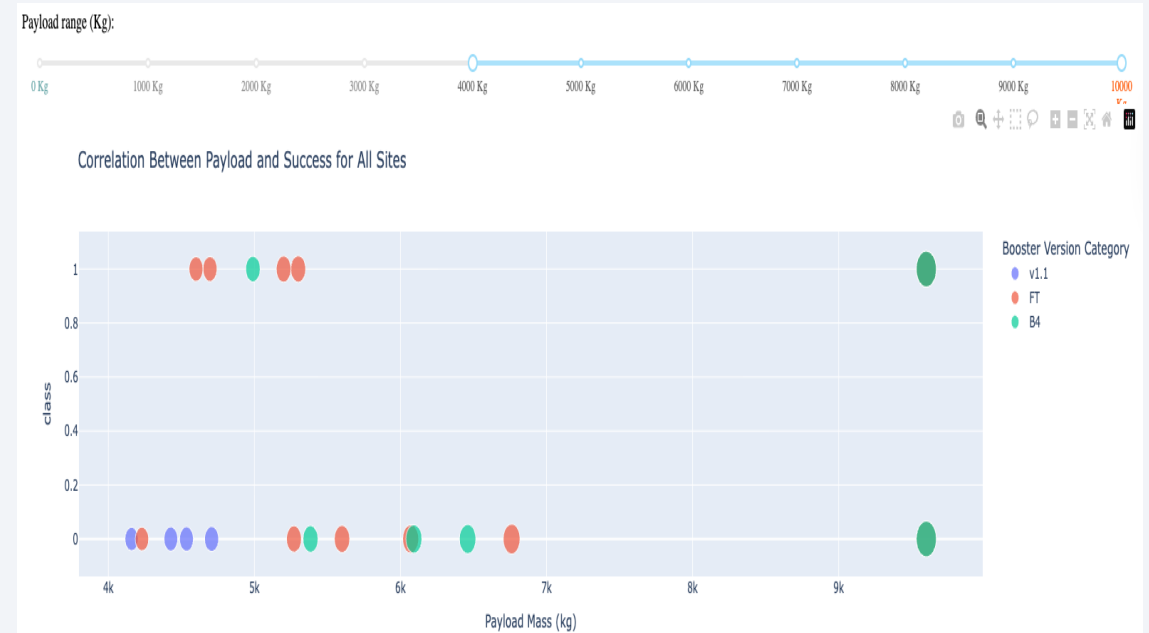
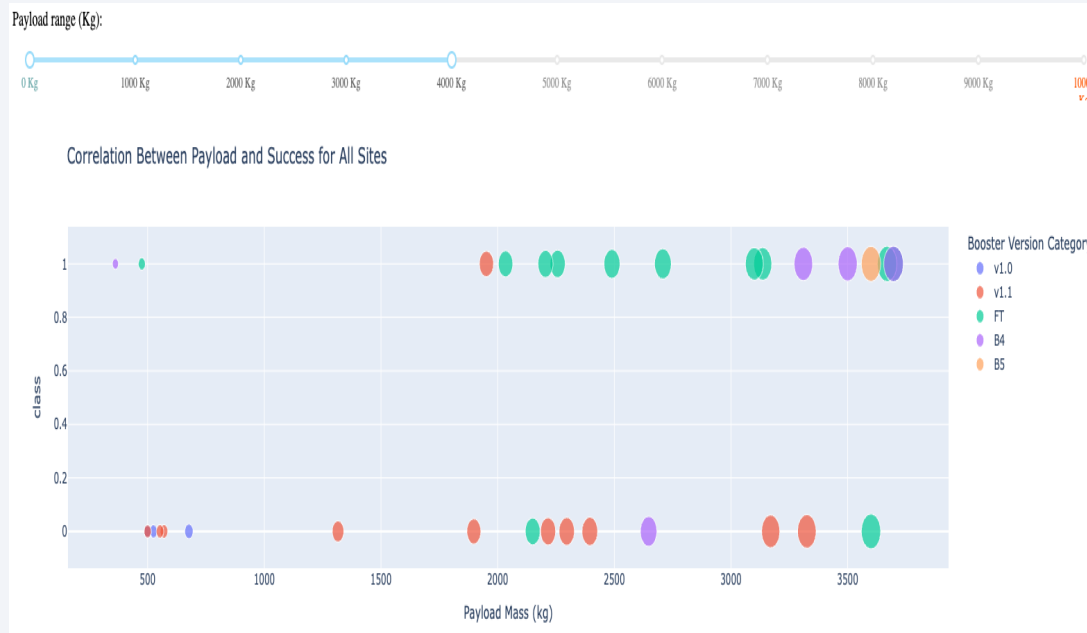
<Dashboard Screenshot 2>

Highest success rate achieved i.e. 76.9% KSC LC-39A with 23.1% failure rate



<Dashboard Screenshot 3>

We can see that all the success rates for low-weighted payload are higher than heavy heavy-weighted payload for both 0kg-4000kg and 4000-10000kg



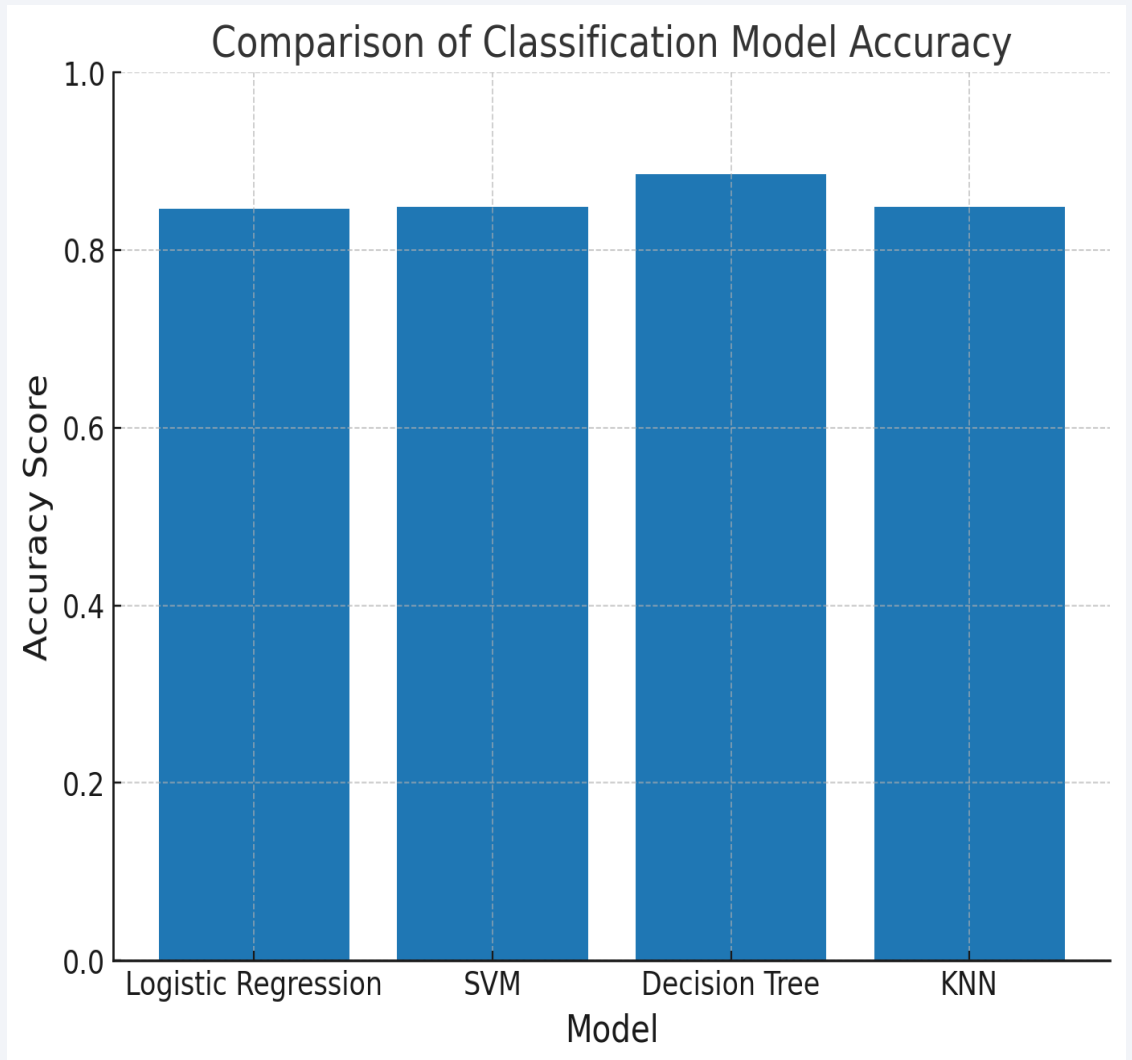
Section 5

Predictive Analysis (Classification)

Classification Accuracy

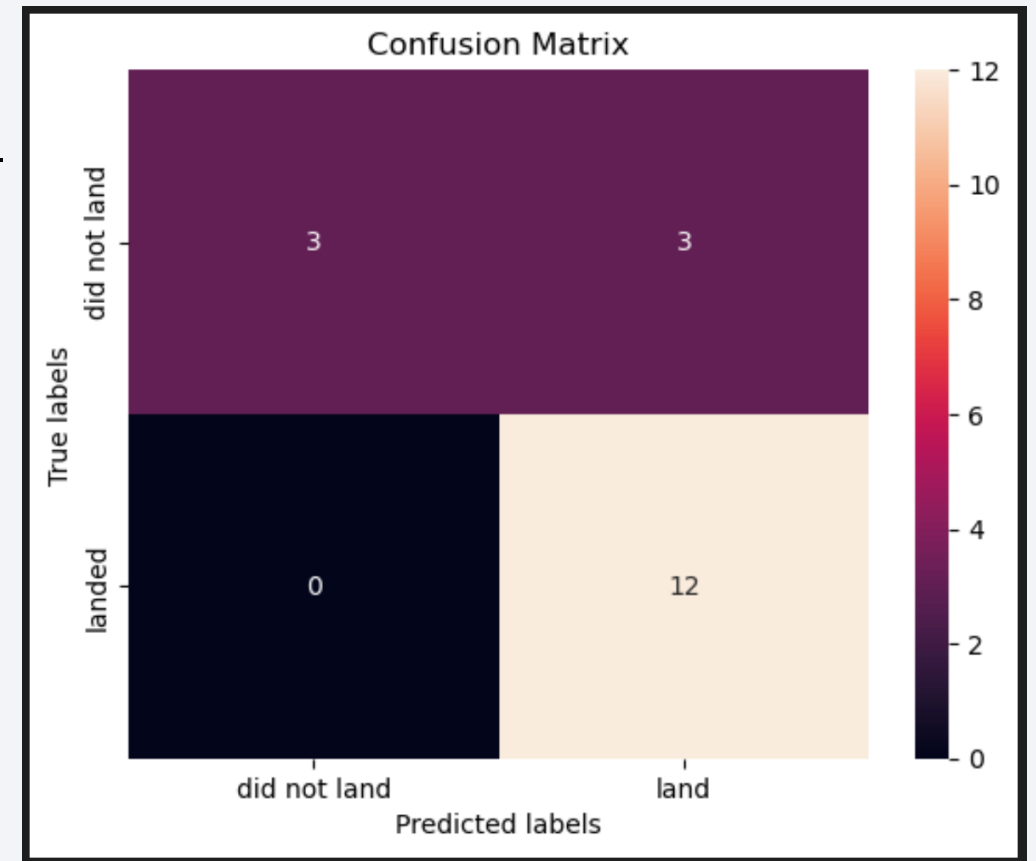
- Model Decision tree with Score 0.875

	Model	Score
0	Logistic regression	0.846429
1	SVM	0.848214
2	Decision tree	0.875000
3	KNN	0.848214



Confusion Matrix

- The **confusion matrix** for the Decision Tree classifier shows that it can effectively distinguish between successful and failed landings. However, the main issue lies in **false positives** — cases where an **unsuccessful landing is incorrectly predicted as successful**.



Conclusions

- The **Decision Tree Classifier** proved to be the **best-performing machine learning model** for this dataset.
- **Light payloads (≤ 4000 kg)** achieved higher success rates compared to heavier payloads.
- Since **2013**, SpaceX launch success rates have steadily increased, trending toward near-perfect outcomes by 2020 and beyond.
- The launch site **KSC LC-39A** recorded the **highest success rate (76.9%)**.
- The **SSO orbit** demonstrated the **highest consistent success rate (100%)** among orbits with more than one occurrence.

Thank you!

